

A Lightweight Causal Evidence Pipeline for Root-Cause Localization in Cloud-Native Systems

Mukesh Dua

Independent Researcher, Seattle, WA, USA
mukesh-dua@outlook.com

Raju Chowdhury

Independent Researcher, Seattle, WA, USA
raju.chowdhury.1384@gmail.com

Aniruddha Singh Gautam

Independent Researcher, Seattle, WA, USA
singhannie@gmail.com

Tarun Kundu

Independent Researcher, Seattle, WA, USA
t.kundu@gmail.com

Abstract—Root-cause localization in cloud-native systems remains challenging because incidents manifest across multiple telemetry signals and no single signal provides sufficient information for unambiguous diagnosis [1] [2] [3]. Existing methods leverage causal graphs and multi-source fusion, yet many require operators to interpret ranked candidates or impact graphs rather than receiving a compact propagation-consistent explanation [4] [5] [6].

We present a lightweight causal evidence pipeline that constructs *interpretable evidence chains*: propagation-aware paths explaining how a fault originated and spread through a service graph. In a small four-service prototype with three injected fault types, the pipeline localized the injected root cause in all 15 tested runs (chain precision 0.83). Under cascade amplification, the no-propagation variant dropped to 0.20 Top-1 (95% CI [0.01, 0.72]) while the proposed ranking maintained 1.00 (95% CI [0.57, 1.00]). These results constitute initial feasibility evidence for the approach, with robustness under noise and partial observability [1] [3] [7].

Index Terms—Root-cause analysis, causal evidence, cloud-native systems, observability, OpenTelemetry, microservices, fault localization

I. INTRODUCTION

Modern observability rests on three pillars (metrics, logs, and traces), whose value grows when linked through shared execution context [1] [2] [8]. Metrics reveal latency and saturation, traces map request paths, and logs capture execution details [1] [3] [7].

Root-cause analysis in microservices has shifted toward graph-based and multi-source reasoning. CloudRanger constructs impact graphs [4], TORAI fuses multi-source telemetry [5], and Nezha performs multi-modal analysis [9]; causal discovery [10] and surveys [6] [11] [12] confirm that structure-aware reasoning outperforms correlation. However, these systems output impact graphs or ranked candidates rather than optimizing a propagation-consistent chain as the primary artifact. While some methods produce path-like explanations (e.g., CloudRanger’s random-walk paths), they do not jointly optimize chain quality via a formal scoring function (Eq. 8) or evaluate chains with a dedicated metric (Chain Precision). Our contribution is this joint optimization-evaluation framework, not multi-source fusion alone.

We propose a compact pipeline that treats **propagation explanation as a measurable inference target**. The cen-

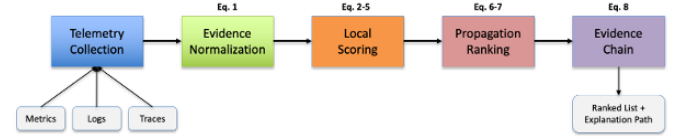


Fig. 1. System overview: telemetry collection, evidence graph construction, ranking, and evidence-chain output.

tral contribution is the **causal evidence chain**: an ordered, propagation-consistent path from root candidate to observed symptoms, with node-level evidence and edge-level propagation confidence (Fig. 1).

Contributions. (1) A unified evidence representation spanning logs, metrics, and traces with formal chain scoring (Eq. 8); (2) a propagation-aware ranking that addresses cases where symptom-heavy propagated services outrank the true root under local scoring; and (3) a reproducible prototype with robustness tests and open-source artifact providing initial evidence for the approach’s viability.

II. METHOD

A. Telemetry Normalization

OpenTelemetry provides a natural basis for unified telemetry because it supports generation and collection of metrics, traces, and logs while propagating trace context across service boundaries [1] [2] [13]. Our pipeline converts each telemetry item into a common evidence tuple:

$$e = (t, v, s, c, a) \quad (1)$$

where t is the timestamp, v is the service, s is the signal type, c is correlation context (trace or span identifiers), and a contains signal-specific attributes such as metric values, log severity, or span latency.

A service dependency graph $G = (V, E)$ is built from trace parent-child relations when available, or from a static deployment map otherwise. Each node $v \in V$ represents a service, and each directed edge $(u, v) \in E$ indicates that service u invokes service v .

B. Local Evidence Scoring

For each service v , we estimate metric anomalies relative to a healthy baseline:

$$z_k(v, t) = \frac{|x_k(v, t) - \mu_k(v)|}{\sigma_k(v) + \epsilon} \quad (2)$$

where $x_k(v, t)$ is the observed value of metric k , $\mu_k(v)$ and $\sigma_k(v)$ are the healthy-baseline mean and standard deviation, and ϵ is a small constant (10^{-8}) that prevents division by zero when baseline variance is negligible.

The local evidence score combines metric, log, and trace signals:

$$E(v, t) = w_m \max_k z_k(v, t) + w_l L(v, t) + w_t T(v, t) \quad (3)$$

where w_m , w_l , and w_t are signal weights (set to 0.4, 0.3, 0.3), $L(v, t)$ is a normalized log evidence score derived from error frequency and severity, and $T(v, t)$ is a trace evidence score derived from span latency inflation and failed spans. Weights were fixed a priori: metric emphasis ($w_m=0.4$) reflects that metrics provide the broadest baseline coverage across services [1], while logs and traces contribute equally (0.3 each) as complementary fault-specific signals. We do not claim these weights are optimal; they are fixed to prevent overfitting and their sensitivity is acknowledged as a limitation.

A practical log evidence term is:

$$L(v, t) = \alpha \cdot \frac{N_{err}(v, t)}{N_{all}(v, t) + \epsilon} + \beta \cdot Sev(v, t) \quad (4)$$

where N_{err} is the error log count, N_{all} is the total log count, Sev is a severity-weighted score, and $\alpha = 0.6$, $\beta = 0.4$ control the relative importance of error frequency versus severity. A practical trace evidence term is:

$$T(v, t) = \delta \cdot \frac{p95_{inc}(v, t) - p95_{base}(v)}{p95_{base}(v) + \epsilon} + \kappa \cdot FailRate(v, t) \quad (5)$$

where the first term captures p95 latency inflation and the second captures the failed-span rate, with $\delta = 0.5$ and $\kappa = 0.5$. Negative inflation is clipped to zero. The scoring window is 5 minutes. Metrics used are CPU utilization, memory usage, and request rate (z-scored against 15-minute baselines). Log severity maps to numeric scores (ERROR=1.0, WARN=0.5, INFO=0.0). C_{call} is normalized by the maximum call count across all edges. When traces are missing, $T(v, t) = 0$ and $C_{trace} = 0$ for affected edges.

C. Propagation-Aware Ranking

Failures in cloud-native systems often ripple to upstream callers, so local anomaly evidence alone may rank symptomatic services above the true source. To account for this, we compute:

$$R(v) = \lambda_1 E(v) - \lambda_2 \sum_{u \in \mathcal{N}^{out}(v)} \rho(v, u) E(u) + \lambda_3 \sum_{p \in \mathcal{N}^{in}(v)} \rho(p, v) E(p) \quad (6)$$

where $\mathcal{N}^{out}(v)$ are services that v calls (downstream callees) and $\mathcal{N}^{in}(v)$ are services that call v (upstream callers), with $\lambda_1=0.5$, $\lambda_2=0.3$, $\lambda_3=0.2$. Since edges point from caller to callee, downstream nodes are potential deeper fault sources. The negative λ_2 term penalizes a service whose callees are also anomalous, while the positive λ_3 term boosts a service whose callers show symptoms, confirming outward propagation.

The edge weight $\rho(\cdot, \cdot)$ captures propagation confidence:

$$\rho(u, v) = \theta_1 C_{trace}(u, v) + \theta_2 C_{time}(u, v) + \theta_3 C_{call}(u, v) \quad (7)$$

where C_{trace} is the fraction of shared trace IDs between u and v (trace continuity), C_{time} is a binary indicator of whether anomalies in u temporally precede those in v , C_{call} is the normalized call frequency from u to v , and $\theta_1=0.4$, $\theta_2=0.3$, $\theta_3=0.3$.

All weight parameters (λ , θ , w , α , β , δ , κ) are fixed a priori and held constant across all scenarios to prevent overfitting to specific fault patterns.

D. Causal Evidence Chains

The ranking (Eq. 6) identifies *which* service is the root cause; a distinct objective recovers *how* the fault propagated. The *call graph* G has edges caller→callee; *fault propagation* flows from the failing callee outward to callers; and the *evidence chain* follows propagation direction (root→symptoms), traversing edges of G^T . The pipeline selects the highest-scoring chain $P^* = \arg \max_P S(P)$ over simple paths in G^T anchored at the top-ranked root:

$$S(P) = \frac{1}{n} \sum_{i=1}^n E(v_i, t_i) + \eta \frac{1}{n-1} \sum_{i=1}^{n-1} \rho(v_i, v_{i+1}) - \gamma \Delta t(P) \quad (8)$$

where n is the number of nodes in path P , η weights mean propagation confidence, γ penalizes time span $\Delta t(P)$ (normalized to $[0, 1]$ by dividing by the 5-minute incident window), and normalization by path length prevents bias toward longer chains. The chain-scoring weights are fixed a priori ($\eta=0.3$, $\gamma=0.1$) and held constant across all scenarios; these values are verifiable in the open-source artifact at tag `v1.0.0` [15]. Ties are broken by shorter path length, then lexicographic service name. This makes evidence-chain extraction a constrained path-optimization problem, distinct from per-node ranking.

The top-scoring chain, anchored at the highest-ranked root cause, provides a human-readable explanation such as `inventory → checkout → frontend`, annotated with supporting signals. When top-candidate scores fall below a fixed confidence gap ($\tau=0.01$), the pipeline flags uncertainty. The chain is optimized (Eq. 8), evaluated (Chain-P), and returned as a first-class RCA output alongside the ranking [5] [4] [6].

III. PROTOTYPE DESIGN

A. Deployment Setup

The prototype uses a small microservice application with four services: `frontend`, `checkout`, `inventory`, and `payment`. Each service emits metrics, structured logs, and

TABLE I
EXPERIMENTAL SCENARIOS

ID	Fault Type	Affected Service	Key Observed Signals
S1	CPU saturation	Inventory	High CPU z-score, error logs, elevated trace latency
S2	Application error	Payment	Exception logs, failed spans, error rate spike
S3	Processing latency fault	Checkout	Span latency inflation, warning logs, upstream timeout propagation

distributed traces, consistent with the multi-signal observability model used by OpenTelemetry [1] [2] [13]. The dependency graph is manually specified to isolate causal reasoning from graph-reconstruction errors; in production, it can be inferred from trace span-parent relationships [7].

The implementation stack consists of Docker Compose, OpenTelemetry SDK and Collector [13], Jaeger for trace storage, Prometheus for metrics, and a Python analysis script for evidence normalization, scoring, ranking, and figure generation.

B. Fault Scenarios

We inject three failure scenarios (Table I), each small enough for a short prototype yet rich enough to produce cross-signal evidence chains:

C. Reproduction Steps

- 1) Run under healthy load for 15 minutes; record per-service baseline statistics (mean, std, p95).
- 2) Export baseline metrics, traces, and logs for all services.
- 3) Inject one fault at a time; collect 5-minute incident telemetry. Repeat each scenario five times.
- 4) Normalize signals into the evidence schema (Eq. 1), build the dependency graph from known topology.
- 5) Compute anomaly scores (Eqs. 2–5), rank services (Eq. 6), extract evidence chains (Eq. 8).
- 6) Compare predicted root cause against true fault location; resolve ties by service name.

IV. RESULTS AND ANALYSIS

A. Quantitative Results

We evaluate signal combinations, reference heuristics, and robustness scenarios across three fault types (S1–S3), each repeated five times. *Chain Precision* (Chain-P) is the fraction of predicted directed propagation edges that appear in the directed ground-truth path; *Chain Recall* (Chain-R) is the fraction of ground-truth directed edges recovered. For the full pipeline, Chain-R = 1.00, Chain-P = 0.83, Chain-F1 = 0.91. Table II reports per-scenario Top-1 (correct/5 runs) and pooled Chain-P.

Reading the table. Max-Z and Metrics-only fail on S1/S2 because metric anomalies propagate across the call graph. Adding traces resolves all scenarios.

TABLE II
PER-SCENARIO TOP-1 ACCURACY (CORRECT/5 RUNS). CHAIN-P = FRACTION OF PREDICTED CHAIN EDGES MATCHING GROUND TRUTH. CASCADE ROWS USE 15× CALLER-SIDE TRACE INFLATION IN S2.

Method	S1	S2	S3	Avg	Chain-P
<i>Reference heuristics</i>					
Max-Z (naive)	0/5	0/5	5/5	0.33	—
PageRank (graph)	0/5	0/5	0/5	0.00	—
No-Propagation	5/5	5/5	5/5	1.00	—
<i>Signal ablations</i>					
Metrics only	0/5	0/5	5/5	0.33	0.67
Metrics + Logs	5/5	5/5	0/5	0.67	0.83
Metrics + Traces	5/5	5/5	5/5	1.00	0.83
Full Pipeline	5/5	5/5	5/5	1.00	0.83
<i>Robustness</i>					
Cascade: No-Propagation	—	1/5	—	0.20	—
Cascade: Full Pipeline	—	5/5	—	1.00	1.00
Partial Obs.	5/5	5/5	5/5	1.00	0.83
Noisy Metrics	5/5	5/5	5/5	1.00	0.83
Degraded Obs.	5/5	2/5	5/5	0.80	0.73

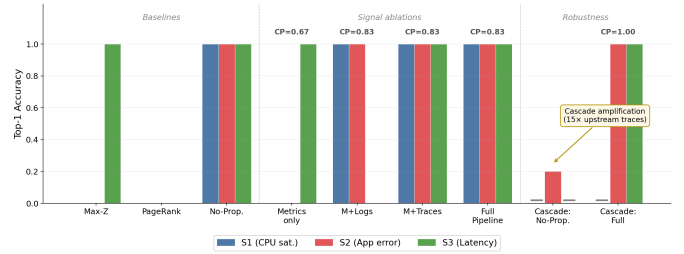


Fig. 2. Per-scenario Top-1 accuracy. The cascade test (right) shows No-Propagation dropping to 0.20 while the full pipeline maintains 1.00. CP = Chain Precision.

Why propagation ranking matters. The cascade stress test amplifies checkout’s trace latencies by 15× in S2. No-Propagation drops to **0.20 Top-1** (1/5 correct; 95% CI [0.01, 0.72]), while the full pipeline maintains **1.00** (5/5; 95% CI [0.57, 1.00]), yielding an accuracy difference of 0.80. While the wide confidence intervals reflect the small sample (5 runs), the consistent failure mode (4/5 misrankings) illustrates the structural weakness of local-only scoring under cascade amplification.

Robustness. Under 0.5σ Gaussian noise, the pipeline retained 1.00 Top-1 and 0.83 Chain-P. A *degraded-observability* test (root-cause logs removed, ± 50 ms trace jitter) maintains 1.00 on S1/S3 but drops to 0.40 on S2, confirming graceful degradation under severe signal loss.

B. Ablation and Baseline Analysis

Anomaly-weighted personalized PageRank [4] scores 0.00 because topology dominates anomaly signals in a small graph; it is included as a graph-only reference heuristic, not a competitive RCA implementation. Given the short-paper scope, our goal is not to outperform mature RCA systems, but to isolate the value of propagation-aware evidence-chain extraction against lightweight and interpretable reference baselines.

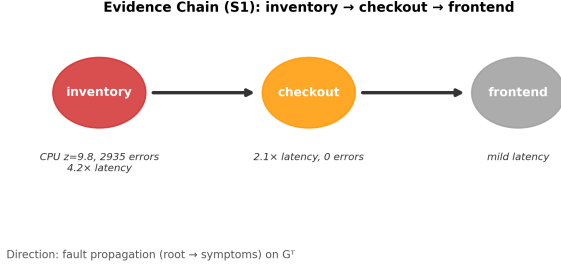


Fig. 3. Evidence chain for S1. Red = root cause, orange = propagation, gray = unaffected. Chain follows G^T (propagation direction).

A direct comparison with full systems (CloudRanger [4], TORAI [5], Nezha [9], MicroRCA [11]) requires standardized benchmarks such as RCAEval [14] and is planned as immediate future work.

C. Case Study

In S1, the pipeline ranked *inventory* first ($E = 80.0$), producing the chain *inventory* → *checkout* → *frontend* on G^T (Fig. 3): *inventory* shows CPU z-score 9.8, 2,935 error logs, and 4.2× latency inflation; *checkout* shows 2.1× latency inflation but zero error logs; *frontend* shows only mild latency elevation. The dominant evidence type shifts along the chain (metric+log at root, trace-only at leaves), demonstrating interpretability.

D. Threats to Validity

The prototype uses four services with synthetic faults and five runs per scenario, limiting statistical power (wide 95% binomial CIs); we do not claim superiority over full RCA systems [4] [5] [9]. Results are feasibility evidence, not proof of global weight optimality. All weights are fixed a priori and verifiable at tag `v1.0.0` [15]. The absence of direct comparison with CloudRanger, TORAI, or MicroRCA is a key limitation; RCAEval [14] evaluation is immediate future work. Manually specified topology means graph-reconstruction errors are untested. Chain-P averages 0.83 due to occasional extra edges (avg. predicted: 2.3 vs. ground truth: 2.0).

V. DISCUSSION

The central finding is that treating propagation explanation as a measurable inference target provides both *robustness* and *explainability* in the tested scenarios [4] [5]. Local evidence scores are computed per service and can be parallelized; chain extraction is bounded by hop limits in larger graphs. Evidence-chain output reduces operator triage to validating a single propagation path [6].

Weight sensitivity. As a lightweight sensitivity check, we perturbed λ , w , and θ by ± 0.1 and observed no Top-1 changes in the three standard scenarios. This does not replace a full sensitivity study, but suggests the prototype is not brittle under small parameter changes.

VI. CONCLUSION

This paper presented a lightweight causal evidence pipeline that treats propagation explanation as a first-class inference target. In this small-scale prototype, the pipeline localized all three injected fault types (Chain-F1 = 0.91), outperforming local-score and graph-only baselines. Under cascade stress, propagation-aware ranking maintained 5/5 correct while No-Propagation dropped to 1/5, though the small sample limits generalization. The causal evidence chain, an optimized, length-normalized, edge-verifiable narrative, is the primary contribution.

Future work will evaluate the pipeline against established RCA methods (CloudRanger [4], TORAI [5], MicroRCA [11]) using the RCAEval benchmark [14] with larger, dynamically inferred service graphs. Additional priorities include stronger baselines, weight sensitivity analysis, additional chain-quality metrics (exact match, Chain-F1 across methods), and learning propagation weights from historical incidents [6]. The full pipeline is available as an open-source artifact [15].

REFERENCES

- [1] OpenTelemetry, “OpenTelemetry Documentation,” 2024. [Online]. Available: <https://opentelemetry.io/docs/>
- [2] OpenTelemetry, “OpenTelemetry Specification,” Cloud Native Computing Foundation, 2024. [Online]. Available: <https://opentelemetry.io/docs/specs/>
- [3] Y. Gan *et al.*, “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems,” in *Proc. ACM ASPLOS*, 2019, pp. 3–18.
- [4] P. Wang *et al.*, “CloudRanger: Root Cause Identification for Cloud Native Systems,” in *Proc. IEEE/ACM CCGrid*, 2018, pp. 492–502.
- [5] L. Pham *et al.*, “TORAI: Multi-Source Root Cause Analysis for Blind Spots in Microservice Service Call Graph,” in *Proc. ACM FSE*, 2026 (to appear).
- [6] J. Soldani and A. Brogi, “Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey,” *ACM Comput. Surv.*, vol. 55, no. 3, 2022.
- [7] B. Sigelman *et al.*, “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google, Tech. Rep., 2010.
- [8] R. Sridharan, *Distributed Systems Observability*, O’Reilly Media, 2018.
- [9] G. Yu *et al.*, “Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-Modal Observability Data,” in *Proc. ACM ESEC/FSE*, 2023.
- [10] A. Ikram *et al.*, “Root Cause Analysis of Failures in Microservices through Causal Discovery,” in *Proc. NeurIPS*, 2022.
- [11] L. Wu *et al.*, “MicroRCA: Root Cause Localization of Performance Issues in Microservices,” in *Proc. IEEE/IFIP NOMS*, 2020.
- [12] M. Li *et al.*, “Going through the Life Cycle of Faults in Clouds: Guidelines on Fault Handling,” in *Proc. IEEE/ACM ICSE-SEIP*, 2023, pp. 121–132.
- [13] OpenTelemetry, “OpenTelemetry Collector,” 2024. [Online]. Available: <https://opentelemetry.io/docs/collector/>
- [14] L. Pham *et al.*, “RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data,” in *Proc. WWW*, 2025.
- [15] M. Dua, “Lightweight Causal Evidence Pipeline for Cloud-Native Observability,” version 1.0.0, 2026. [Online]. Available: <https://github.com/mukesh-dua/causal-evidence>